

REVIEW 1

Database Systems Engineering and Distributed
Backend Development
Course Code: 25CS13O2E

CIPHERCHAT

CHAT APPLICATION WITH
END-TO-END ENCRYPTION

TEAM MEMBERS:

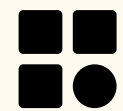
CHAKRIKA PANAKALA — 2520090042
AMRUTHA JAVVADI — 2520080066
HRUSHIKA HABEEB — 2520090054



Problem Statement

- Traditional messaging systems rely on centralized servers to process and store messages.
- Server-side access to plaintext messages creates privacy and security concerns.
- Users need secure communication where only the sender and intended recipient can read messages.
- Existing secure messaging solutions can be complex, while privacy and usability need to coexist.

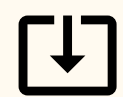
Objectives



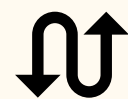
Ensure End-to-End Encryption



Protect Media Sharing



Secure User Authentication



Support Message Synchronization



Enable Real-Time Messaging



Maintain Privacy





System Features



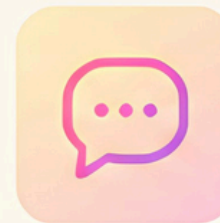
End-to-End Encrypted

Only you and your recipient can read messages.



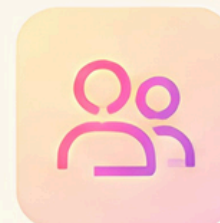
Private & Secure

Your privacy is our top priority.



Simple & Seamless

Secure messaging made easy.

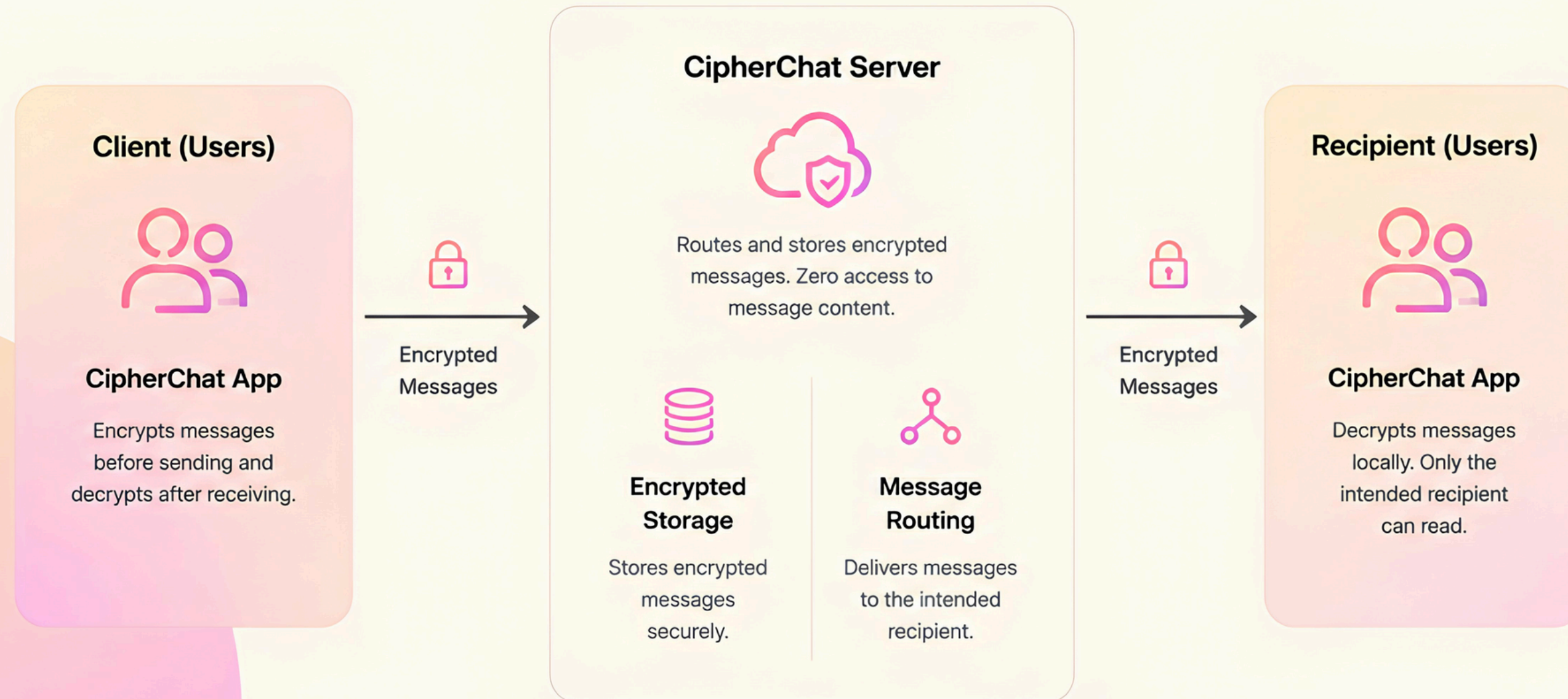


No Data Stored

We don't store your messages or personal data.

System Architecture

Built for security, privacy, and simplicity.



Key Principles

- End-to-End Encryption**
Messages are encrypted on the sender's device and decrypted only on the recipient's device.
- Zero Knowledge Server**
Our server never sees your messages or keys.
- Privacy First**
No data is stored in plain text. Your privacy is our priority.
- Secure & Reliable**
Designed for secure delivery, high availability, and a seamless experience.



Encrypted Data Flow



Secure Server Components

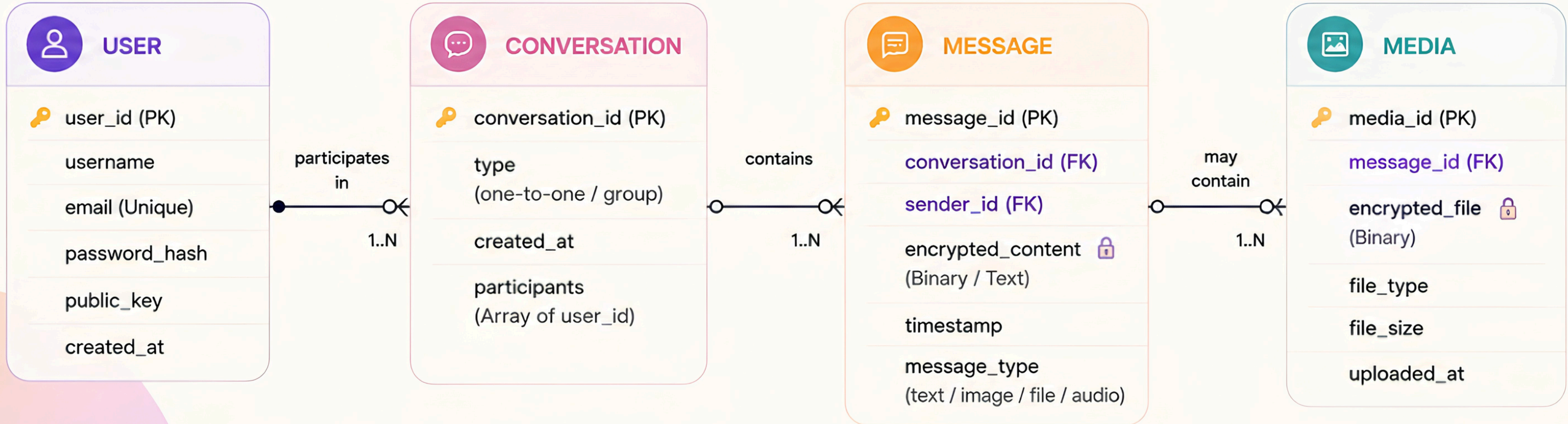


User Devices

Database Design

ER Diagram / Database Schema

All sensitive content is encrypted before storage. MongoDB stores only encrypted data and metadata.



Relationships

- User → participates in → Conversation
- User → sends → Message
- Conversation → contains → Messages
- Message → may contain → Media



Security & Privacy

- ✓ End-to-end encryption ensures only the intended recipient can decrypt messages.
- ✓ MongoDB stores only encrypted message content and metadata.
- ✓ Plaintext messages are never stored on the server.



Tech Stack



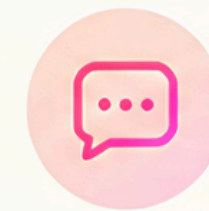
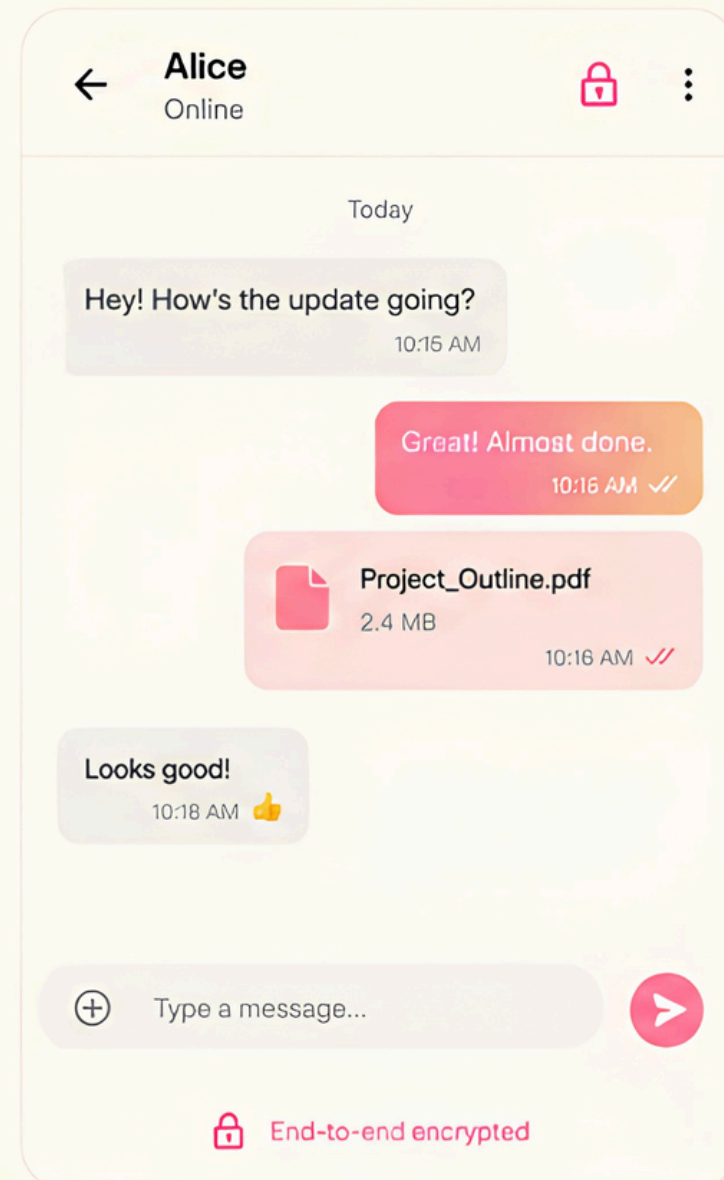
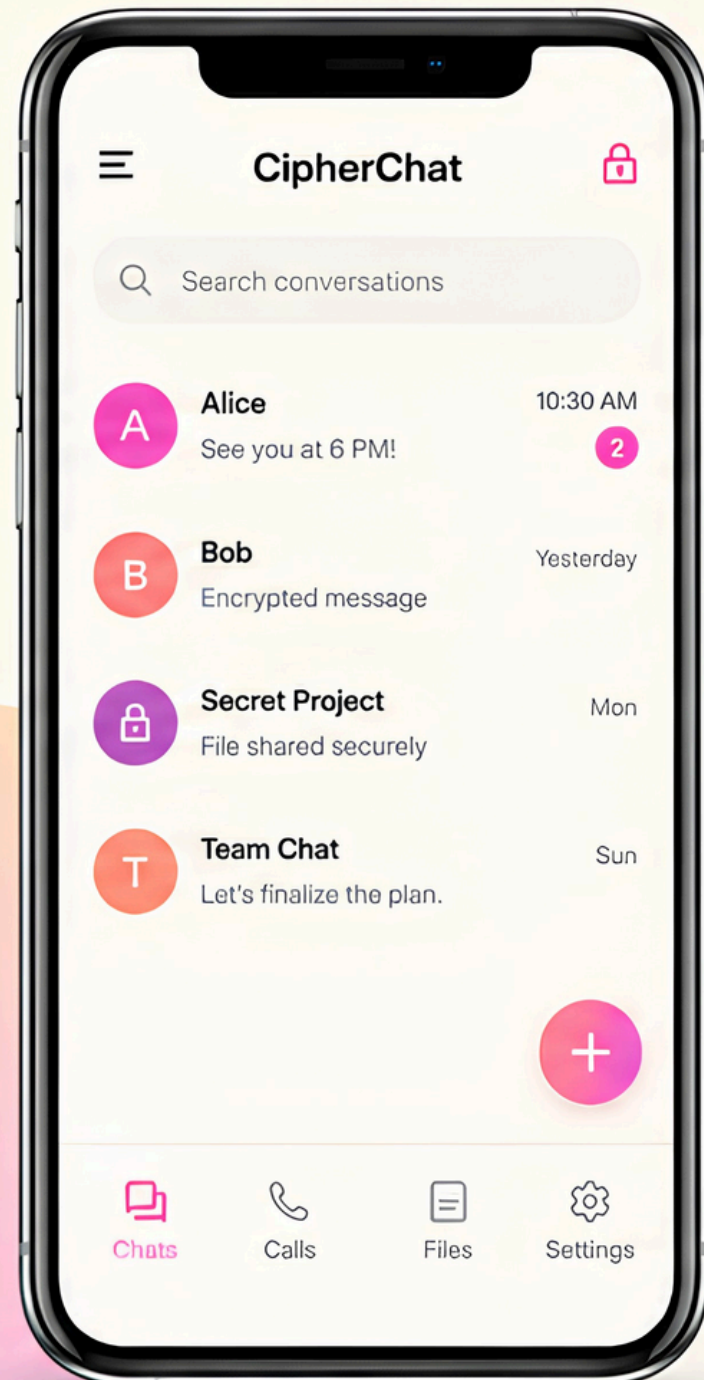
MongoDB
(Encrypted Data at Rest)



Node.js
(Backend API)

Front-End Implementations

Intuitive. Modern. Secure.



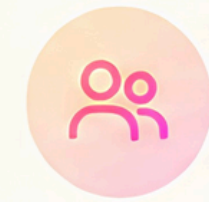
Clean & Minimal UI

A distraction-free interface that focuses on secure communication.



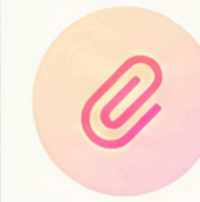
Secure by Design

Visual indicators for encryption status and secure actions across the app.



Smooth Navigation

Easy access to chats, calls, files, and settings with an intuitive bottom navigation.



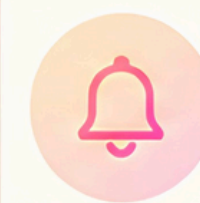
Media & File Sharing

Securely send files, images, audio, and documents encrypted end-to-end.



Customizable Themes

Light & dark mode support with modern, soft gradient accents.



Real-time Updates

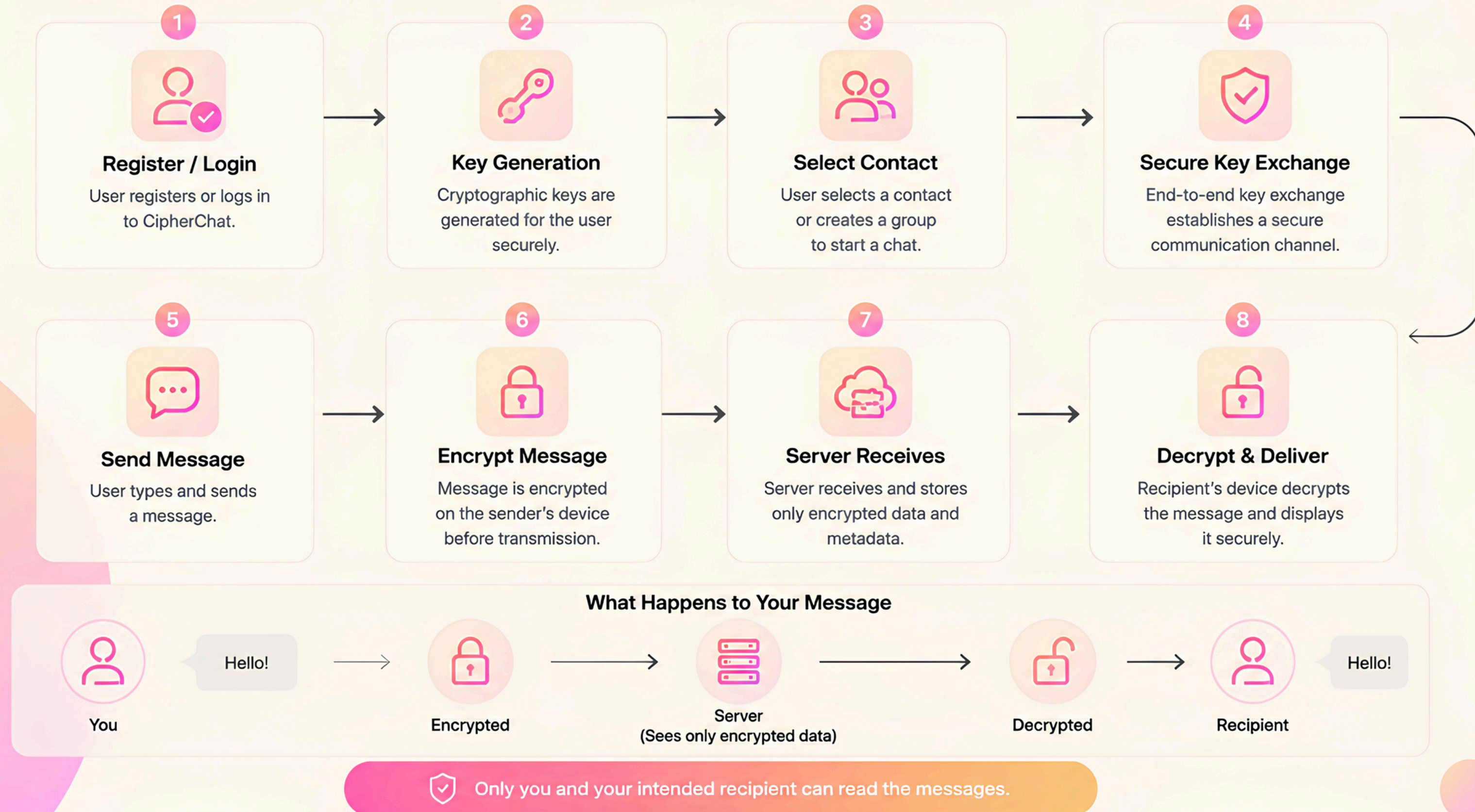
Instant message delivery and real-time indicators for a seamless experience.



Every interaction is encrypted.
Every experience is seamless.

Demo Flow

A seamless journey from login to secure messaging.



Future Scope

Building the future of secure communication.



Mobile Application

Extend the platform to Android/iOS.



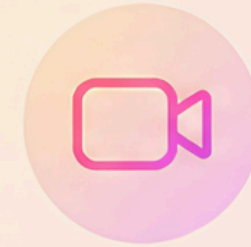
Advanced Security

Improve key rotation, device verification and session management.



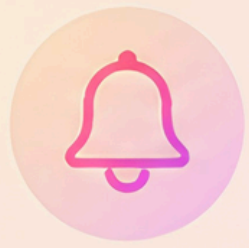
Secure Cloud Backup

Enable encrypted backups across devices.



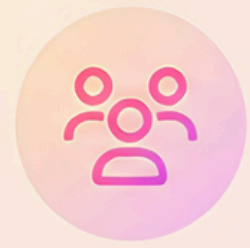
Voice & Video Calling

Add encrypted real-time audio/video communication.



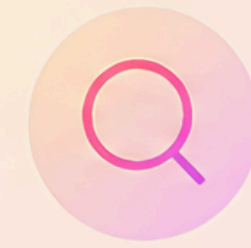
Push Notifications

Notify users of new messages securely.



Larger Group Communication

Optimize encrypted messaging for large groups.



Security Auditing

Conduct penetration testing and cryptographic security reviews.

Conclusion

Summarizing our work and its impact.



Developed a real-time chat application focused on privacy and secure communication.



Implemented end-to-end encryption
so the server does not access plaintext messages.



Integrated React.js, Node.js, Socket.io, MongoDB and libsodium
into a distributed application architecture.



Demonstrates how database systems, real-time communication
and modern cryptography can be combined to create a privacy-first
messaging platform.

Secure communication without compromising usability.



Thank You